American International University-Bangladesh

Faculty of Science and Technology

PROJECT REPORT

on

Student Performance Analysis: Predicting Academic Outcomes with Machine Learning

Faculty: MD. TANZEEM RAHAT
Course: Programming in Python

Submitted By

Student ID	Name
23-51629-2	Kaif Khan
23-51633-2	Kowshik Kormokar
23-50780-1	Istehadul Islam
22-48825-3	Kamrul Hasan Tonmoy

Table of Contents

1. Introduction

Predicting how well a student is likely to perform is something teachers and academic advisors are often interested in, since knowing this early can help them decide who might need extra support. This project looks at that question using machine learning. Instead of building an application with a user interface, the team worked with an existing dataset of secondary school students and built a Python based analysis pipeline that cleans the data, explores it, and then trains several classification models to predict a student's overall performance category.

The dataset used is the well known UCI Student Performance dataset for a mathematics course, which records 395 students from two Portuguese secondary schools. Each student has 33 attributes describing their background (such as parents' education and job, family size, and whether they live in an urban or rural area), their study habits and social life (study time, going out with friends, alcohol consumption, free time), and their academic record across three grading periods, labelled G1, G2 and G3.

Rather than predicting the exact final grade, the project turns G3 into three performance bands, Low, Medium and High, and treats the problem as a classification task. A majority class baseline, a K Nearest Neighbors model, a Support Vector Machine, and a Random Forest classifier are all trained on the same data so their results can be compared fairly. The project was carried out as part of the Programming in Python course, and gave the team a chance to practice data cleaning, exploratory analysis, building reusable preprocessing pipelines with scikit learn, and evaluating models properly instead of just reporting a single accuracy number.

2. Objectives

- To explore the UCI student performance dataset through descriptive statistics and visualizations, and check it for missing values, duplicates, or unusual entries.
- To turn the final grade (G3) into a three level performance category, Low, Medium and High, that can be used as a classification target.
- To build a single, reusable preprocessing pipeline with scikit learn's ColumnTransformer that scales numeric features and one hot encodes categorical features.
- To set up a simple majority class baseline so the more advanced models have something meaningful to be compared against.
- To tune and evaluate a K Nearest Neighbors classifier and a Support Vector Machine using cross validation and grid search.
- To train a Random Forest classifier and look into its predictions further through feature importance and misclassification analysis.
- To compare every model on accuracy, macro precision, macro recall and macro F1 score, since the three performance classes are not equally represented in the data.

3. Tools and Technologies Used

Tool / Library	Purpose
Python 3	Core programming language used for the whole project
pandas	Loading the CSV dataset and handling it as a DataFrame throughout the pipeline
NumPy	Underlying numeric operations used by pandas and scikit learn
scikit learn	Preprocessing (scaling, one hot encoding), model building (Dummy, KNN, SVM, Random Forest), cross validation, grid search and evaluation metrics
matplotlib and seaborn	Plotting distributions, boxplots, correlation heatmaps, confusion matrices and the feature importance chart
joblib	Saving the trained Random Forest model to disk for reuse
Jupyter Notebook	Used for the exploratory data analysis and the modelling walkthroughs

4. Project Structure

The project separates the exploratory notebooks from the reusable pipeline code, so the logic used for loading data and preprocessing it does not have to be rewritten inside every notebook. The overall layout is shown below.

```
student-performance-analysis-main/
├── main_random_forest_pipeline.py
├── RandomForestModel.py
├── ModelEvaluator.py
├── ErrorAnalyzer.py
├── FeatureInterpreter.py
├── data/
│   └── raw/
│       └── student-mat.csv
├── src/
│   ├── data_loader.py
│   ├── preprocessing.py
│   ├── baseline.py
│   ├── knn_model.py
│   └── svm_model.py
├── notebooks/
│   ├── 01_data_audit_eda.ipynb
│   ├── 02_preprocessing_baseline.ipynb
│   └── 03_knn_svm.ipynb
├── figures/
└── results/
```

Figure 1: Folder and file structure of the project

This keeps a clear separation of responsibilities. The data folder holds the raw dataset, src holds the code that turns that raw CSV into a model ready set of features and labels, the top level

scripts (RandomForestModel.py, ModelEvaluator.py, ErrorAnalyzer.py, FeatureInterpreter.py) handle training and inspecting the Random Forest, notebooks documents the exploratory work and the KNN and SVM experiments step by step, and figures and results hold everything the pipeline produces, the saved plots and the CSV score tables respectively.

5. Module Description

5.1 src/data_loader.py

Handles reading the raw dataset. load_student_data() points to data/raw/student-mat.csv, reads it with pandas using a semicolon separator (the format the UCI file actually uses), and raises a FileNotFoundError with a clear message if the file is missing. validate_columns() checks the loaded DataFrame against the 33 expected column names, raising an error if any are missing and printing a warning if unexpected extra columns show up.

5.2 src/preprocessing.py

This is where the raw grades turn into a classification problem. create_target() bins the final grade G3 into three bands, Low for 0 to 9, Medium for 10 to 14, and High for 15 to 20, and stores this as a new performance_class column. create_features_and_target() then splits the DataFrame into a feature matrix (14 numeric columns such as age, study time, failures and the two earlier grades G1 and G2, plus 17 categorical columns such as school, sex, address and parents' jobs) and the target series. build_preprocessor() wires these into a scikit learn ColumnTransformer that applies StandardScaler to the numeric columns and OneHotEncoder to the categorical ones, so both model types can consume the same feature matrix.

5.3 src/baseline.py

build_baseline() wraps the preprocessor together with scikit learn's DummyClassifier set to the most_frequent strategy. This gives a simple reference point: a model that always predicts the most common class, which any of the real models should be able to beat comfortably.

5.4 src/knn_model.py

build_knn() builds a pipeline combining the preprocessor with a KNeighborsClassifier. select_best_k() goes a step further and runs five fold stratified cross validation for a range of k values (3, 5, 7, 9 and 11), scoring each with macro F1 rather than plain accuracy, since the three performance classes are not evenly sized. It returns a small results table along with whichever k scored best on average.

5.5 src/svm_model.py

build_svm() builds a pipeline with an SVC classifier. tune_svm() runs a grid search with five fold stratified cross validation over two families of settings, an RBF kernel with different values of C and gamma, and a linear kernel with different values of C, again scored on macro F1. It returns the fitted grid search object along with a table of every combination that was tried.

5.6 RandomForestModel.py, ModelEvaluator.py, ErrorAnalyzer.py, FeatureInterpreter.py

These four top level scripts form the Random Forest side of the project, tied together in main_random_forest_pipeline.py. RandomForestModel wraps a RandomForestClassifier (200 trees, balanced class weights) with fit and predict methods, plus a save_model() helper built on joblib. ModelEvaluator takes the true and predicted labels and produces a classification report as a DataFrame, and can also plot and save a confusion matrix heatmap with seaborn. ErrorAnalyzer pulls out the test rows the model got wrong and writes them to results/error_analysis.csv, so the misclassifications can be looked at more closely later. FeatureInterpreter reads the trained model's feature_importances_ and plots them as a sorted horizontal bar chart, saved to figures/feature_importance_rf.png.

5.7 notebooks/

01_data_audit_eda.ipynb covers the initial data audit and exploratory analysis, including checks for missing values and duplicates, boxplots for grades, age and absences, and scatter plots relating study habits and failures to the final grade. 03_knn_svm.ipynb loads the data through the src modules, runs the k selection and SVM grid search described above, evaluates both models on the held out test set, and saves their confusion matrices and score tables. 02_preprocessing_baseline.ipynb was set up for the baseline experiments but the team ended up moving that logic into src/baseline.py directly.

6. Workflow of the System

Stage	Description
Data loading and validation	Reads student-mat.csv and confirms all 33 expected columns are present before anything else runs
Exploratory data analysis	Checks for missing values and duplicates, then visualizes grade distributions, boxplots and correlations between features and the final grade
Target creation	Converts the continuous final grade G3 into the Low, Medium, High performance classes used for classification
Preprocessing	Scales the 14 numeric features and one hot encodes the 17 categorical features through a single ColumnTransformer
Baseline model	Trains a majority class DummyClassifier as a minimum bar for the other models to clear
KNN tuning and evaluation	Selects the best k by cross validated macro F1, then evaluates the chosen model on the test set
SVM tuning and evaluation	Grid searches kernel, C and gamma by cross validated macro F1, then evaluates the best model on the test set
Random Forest, evaluation and interpretation	Trains a Random Forest, evaluates it, saves misclassified rows for review, and plots feature importance
Results export	Saves each model's scores as CSV files in results/ and every plot as a PNG file in figures/

7. Classification Logic

The classification target is derived directly from the final grade G3, which in the original dataset runs from 0 to 20. The project groups this range into three performance bands, shown below, so that predicting a student's outcome becomes a three class problem instead of a regression on the exact grade.

Final Grade (G3)	Performance Class	Students in Dataset
0 - 9	Low	130
10 - 14	Medium	192
15 - 20	High	73

Out of 395 students, close to half fall into the Medium band while High performers make up less than a fifth of the class, so the dataset is noticeably imbalanced. This is exactly why the project scores its models with macro precision, macro recall and macro F1 alongside plain accuracy, since a model could otherwise look good just by favoring the Medium class and still do a poor job of recognizing the smaller High and Low groups.

8. Sample Data and Output

The raw dataset is a semicolon separated CSV file where every row is one student, described by 32 background and behavioral attributes plus their three grades. A shortened excerpt of one row is shown below.

```
school="GP"; sex="F"; age=18; address="U"; famsize="GT3";
Medu=4; Fedu=4; Mjob="at_home"; Fjob="teacher";
studytime=2; failures=0; absences=6;
G1=5; G2=6; G3=6  ->  performance_class = Low
```

After the models were trained and tested on a stratified 80/20 split of the data, the following scores were recorded on the held out test set.

Model	Accuracy	Macro Precision	Macro Recall	Macro F1
Baseline (Dummy)	48.10%	-	-	-
K Nearest Neighbors (k = 9)	60.76%	62.07%	59.47%	60.14%
SVM (linear kernel, C = 10)	82.28%	81.95%	83.81%	82.69%

The Support Vector Machine with a linear kernel came out clearly on top once tuned, improving on the KNN result by more than twenty percentage points and roughly doubling the majority class baseline. This lines up with what the exploratory analysis had already suggested, since the two earlier grades, G1 and G2, are strongly related to the final grade, and a linear decision boundary over the scaled features seems to capture that relationship well. Alongside these tables, the pipeline also saves a set of figures, including boxplots for G1, G2, G3, age and absences, scatter plots of study time, failures and absences against the final grade, a correlation heatmap, the KNN k selection curve, and confusion matrices for the baseline, KNN and SVM models.

9. Testing and Error Handling

Data loading is checked before any modelling begins: load_student_data() raises a FileNotFoundError with the exact expected path if the CSV is missing, and validate_columns() raises a ValueError listing any missing columns, which stops the pipeline early rather than letting a malformed dataset silently produce wrong results.

Because the performance classes are imbalanced, every train and test split, along with every cross validation fold used for tuning KNN and SVM, is stratified, so the Low, Medium and High classes stay represented in roughly the same proportions throughout. This avoids a fold accidentally ending up with almost no High performing students, which would make the reported scores unreliable.

On the Random Forest side, ErrorAnalyzer specifically isolates the rows the model got wrong on the test set and saves them, with both the true and predicted label attached, to results/error_analysis.csv. This makes it possible to go back afterward and look for patterns among the students the model struggled with, rather than only seeing a single aggregated accuracy figure.

10. Challenges Faced

1. The performance classes are not balanced, Medium students outnumber High students by almost three to one, which meant plain accuracy alone was not a trustworthy way to judge a model, and the team had to lean on macro precision, recall and F1 instead.

2. Deciding where to draw the cut offs between Low, Medium and High needed some thought, since the boundaries had to reflect a meaningful academic distinction rather than just splitting the range into equal thirds.

3. Tuning KNN's k and SVM's kernel, C and gamma took a fair amount of trial and error, and the team had to be careful that the cross validation setup did not let information from the test set leak into the tuning process.

4. Keeping the exploratory notebooks and the reusable src modules consistent with each other was harder than expected. Logic that started out in 02_preprocessing_baseline.ipynb eventually moved into src/baseline.py, and the notebook itself was left mostly empty.

5. Coordinating four group members around a shared data loading and preprocessing pipeline meant agreeing early on how the target and features should be structured, so that the KNN, SVM and Random Forest components could all plug into the same pipeline without conflicts.

11. Future Enhancements

1. Try gradient boosting models such as XGBoost or LightGBM alongside the existing KNN, SVM and Random Forest classifiers, since they often do well on tabular data like this.

2. Run a proper hyperparameter search for the Random Forest (number of trees, max depth, minimum samples per leaf) the same way KNN and SVM were tuned.

3. Add learning curves and cross validated confidence intervals so the reported scores come with a sense of how stable they are, not just a single number from one split.

4. Wrap the trained pipeline behind a small command line tool or web form where a new student's details could be entered to get a predicted performance class.

5. Use SHAP values instead of, or alongside, the current feature importance chart, to explain individual predictions rather than only the model's overall feature ranking.

12. Conclusion

This project gave the team a chance to work through a complete, realistic machine learning workflow in Python, starting from an untouched CSV file and ending with several trained and compared classification models. The UCI student performance dataset turned out to be clean, with no missing values or duplicates, and previous academic performance clearly stood out as the strongest signal for predicting a student's final outcome, well ahead of study time or attendance. Of the models tried, the majority class baseline set the floor at just under half of the test set correct, K Nearest Neighbors pushed that closer to sixty one percent once tuned, and the linear Support Vector Machine performed best by a wide margin, correctly classifying just over eighty two percent of students on data it had not seen during training. Beyond the specific numbers, the project reinforced practical skills that go beyond any one model, building reusable preprocessing pipelines, handling class imbalance sensibly, validating input data before it reaches a model, and evaluating results with more than a single accuracy figure.

13. References

- Cortez, P. and Silva, A. (2008). Using Data Mining to Predict Secondary School Student Performance. UCI Machine Learning Repository, Student Performance Data Set.
- scikit learn developers. scikit learn: Machine Learning in Python. https://scikit-learn.org/
- The pandas development team. pandas documentation. https://pandas.pydata.org/
- Hunter, J.D. Matplotlib: A 2D Graphics Environment. https://matplotlib.org/
- Waskom, M. seaborn: statistical data visualization. https://seaborn.pydata.org/